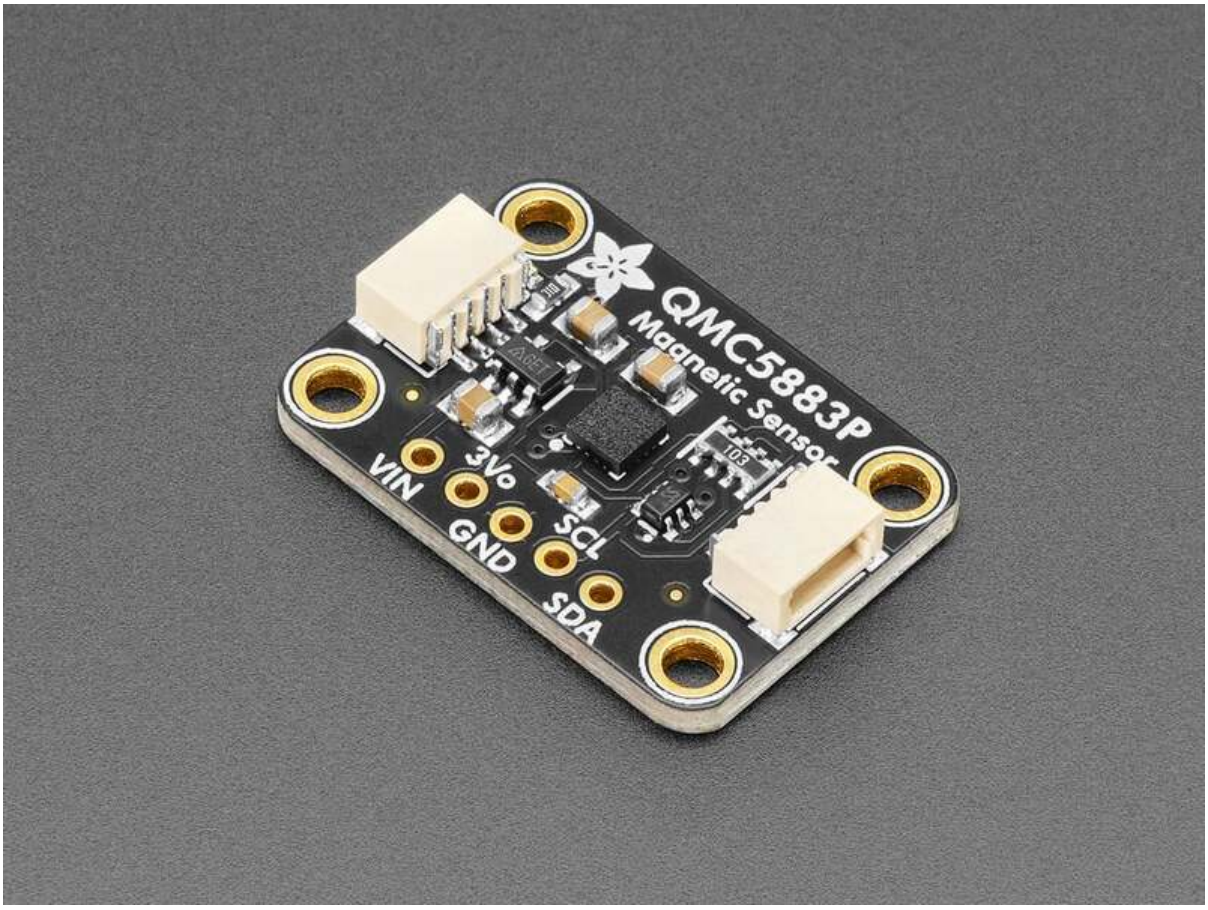
Adafruit QMC5883P - Triple Axis Magnetometer

Created by Liz Clark



https://learn.adafruit.com/adafruit-qmc5883p-triple-axis-magnetometer

Last updated on 2025-09-29 10:22:24 AM EDT

Table of Contents

Overview 3

Pinouts 5

- Power Pins
- I2C Logic Pins
- Power LED and Jumper

CircuitPython and Python 6

- CircuitPython Microcontroller Wiring
- Python Computer Wiring
- Python Installation of QMC5883P Library
- CircuitPython Usage
- Python Usage
- Example Code

Python Docs 10

Arduino 10

- Wiring
- Library Installation
- Example Code

Arduino Docs 15

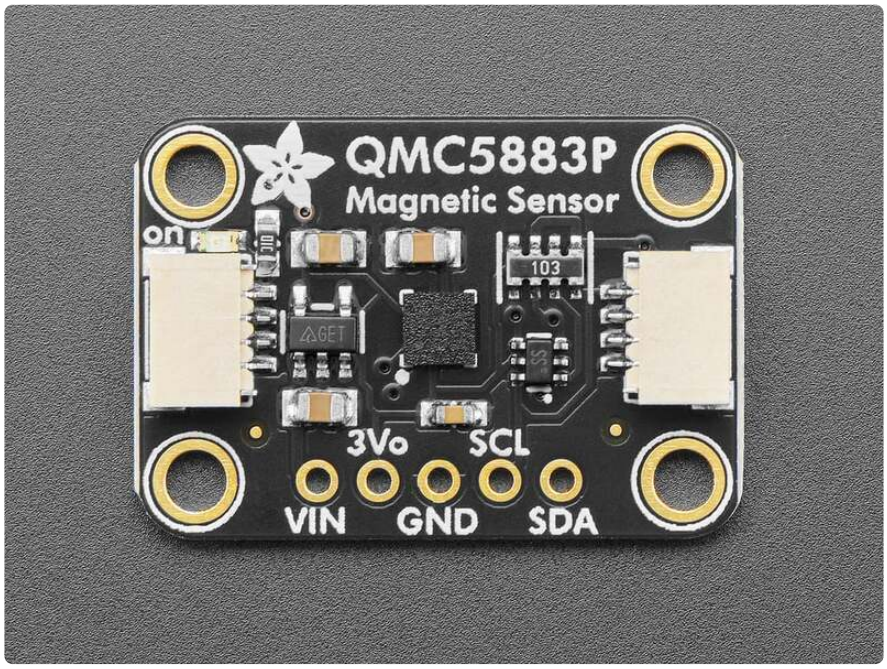
WipperSnapper 16

- What is WipperSnapper
- Wiring
- Usage

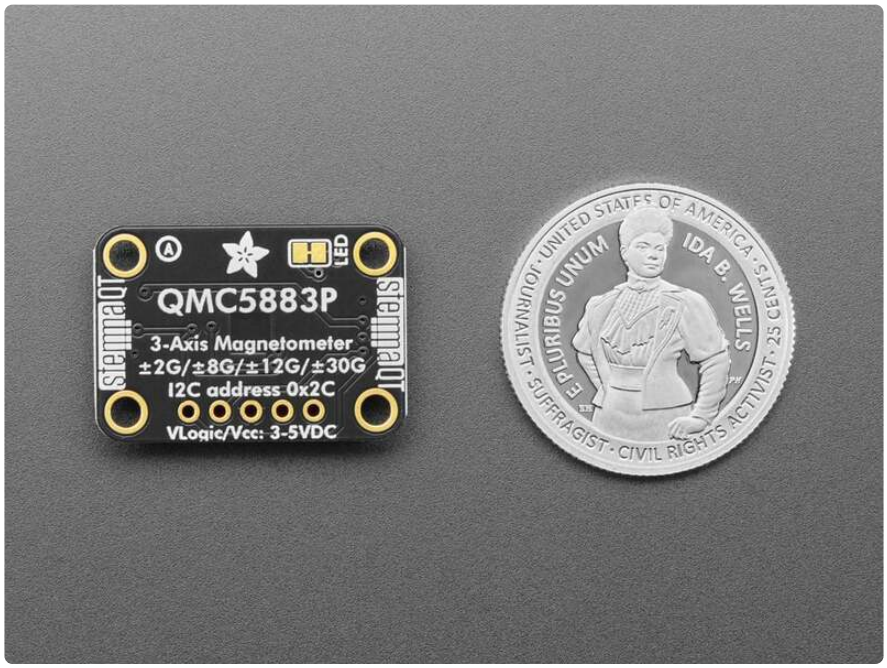
Downloads 22

- Files
- Schematic and Fab Print

Overview



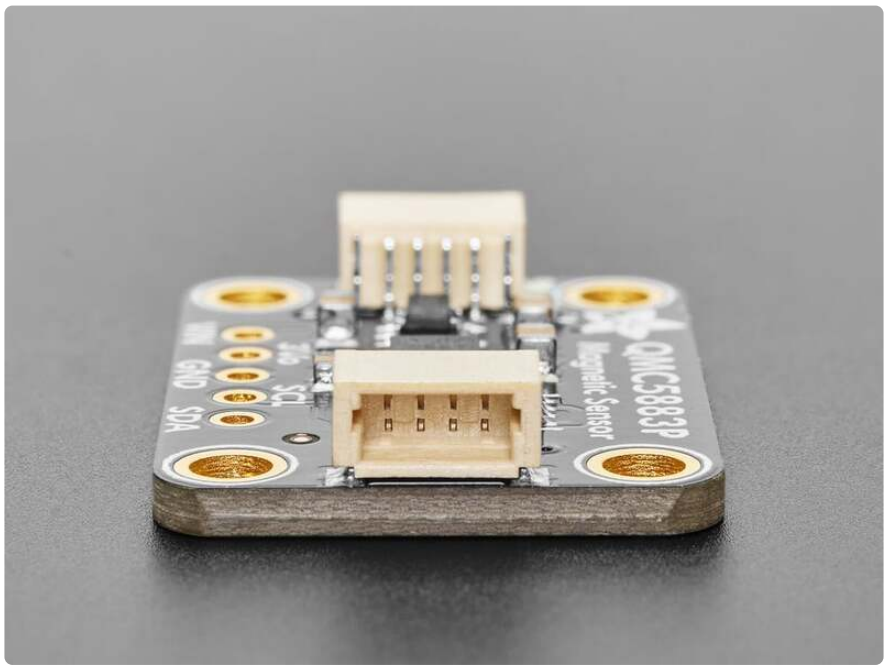
Sense the magnetic fields that surround us with this handy triple-axis magnetometer (compass) module. Magnetometers can sense where the strongest magnetic force is coming from, generally used to detect magnetic north, but can also be used for measuring magnetic fields. This sensor tends to be paired with a 6-DoF (degree of freedom) accelerometer/gyroscope to create a 9-DoF inertial measurement unit that can detect its orientation in real-space, thanks to Earth's stable magnetic field.



We based this breakout on the QMC5883P (https://adafru.it/1apd), a magnetometer we see used in many Quadcopter and robotic builds. This compact sensor uses I2C to

communicate, and it's very easy to use. Simply download our library and connect the SCL pin to your I2C clock pin, and SDA pin to your I2C data pin and upload our test program to read out magnetic field data.

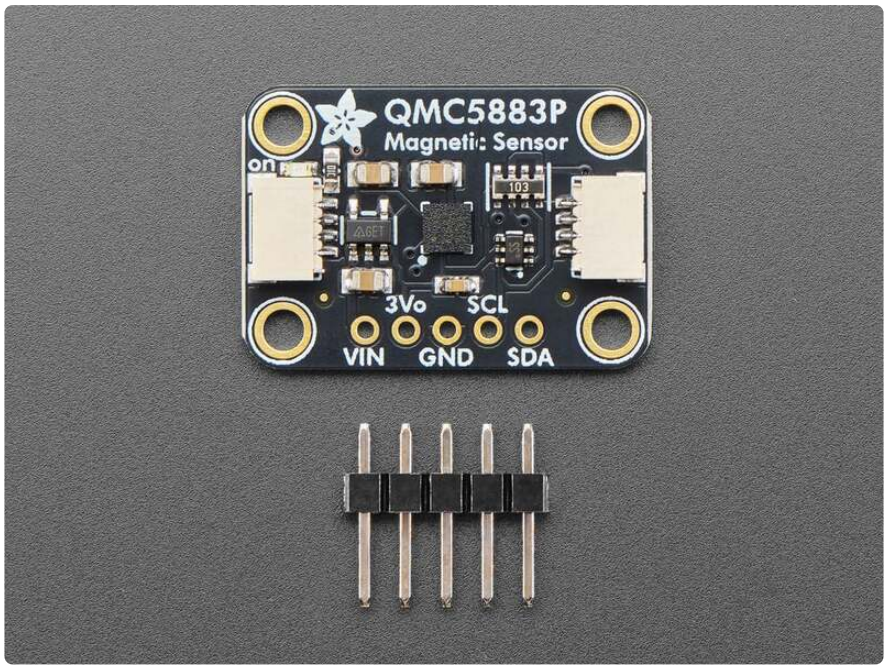
This sensor can sense ranges from +-2 Gauss (+- 200 uTesla) up to +-30 gauss (+-3000 uT or 3.0 mT). With 16-bit ADC and settings for over-sampling and downsampling, you can set it up to get just the right range and precision.



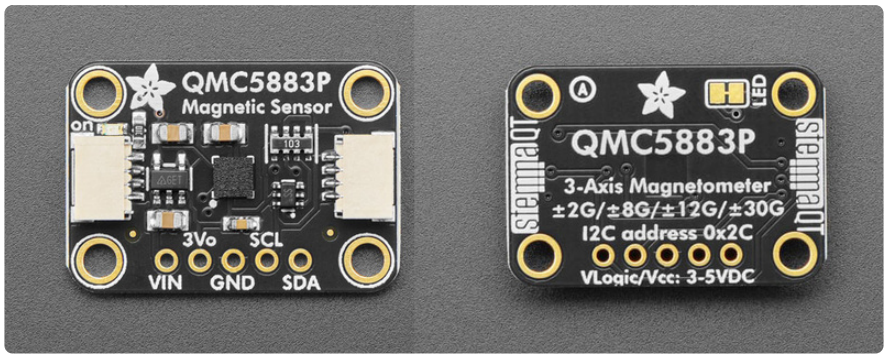
To make life easier so you can focus on your important work, we've taken the QMC5883P and put it onto a breakout PCB along with support circuitry to let you use this little wonder with 3.3V (Feather/Raspberry Pi) or 5V (Arduino/ Metro328) logic levels. Additionally, since it speaks I2C, you can easily connect it up with two wires (plus power and ground!).

We've even included SparkFun qwiic (https://adafru.it/Fpw) compatible STEMMA QT **(https://adafru.it/Ft4)** connectors for the I2C bus so **you don't even need to solder!** Just wire it up to your favorite micro and you can use our CircuitPython/Python or Arduino drivers to easily interface with the QMC5883P (https://adafru.it/1ape) and get magnetic measurements ASAP. **QT Cable is not included**, but we have a variety in the shop (https://adafru.it/17VE).

It's fully assembled and tested. Comes with a bit of 0.1" standard header in case you want to use it with a breadboard or perfboard. Four 2.5mm (0.1") mounting holes for easy attachment.



Pinouts



The default I2C address is **0x2C**.

Power Pins

- **VIN** - this is the power pin. It can be powered by 3V or 5V. Give it the same power as the logic level of your microcontroller - e.g. for a 5V micro like Arduino, use 5V.
- **3Vo** - this is the 3.3V output from the voltage regulator, you can grab up to 100mA from this if you like.
- **GND** - common ground for power and logic.

I2C Logic Pins

- **SCL** - I2C clock pin, connect to your microcontroller's I2C clock line. This pin can use 3-5V logic, and there's a **10K pullup** on this pin.
- **SDA** - I2C data pin, connect to your microcontroller's I2C data line. This pin can use 3-5V logic, and there's a **10K pullup** on this pin.

- STEMMA QT **(https://adafru.it/Ft4)** - These connectors allow you to connect to development boards with **STEMMA QT** (Qwiic) connectors or to other things with various associated accessories (https://adafru.it/Ft6).

Power LED and Jumper

- **Power LED** - In the upper left corner, above the STEMMA connector, on the front of the board, is the power LED, labeled **on**. It is a green LED.
- **LED jumper** - This jumper is located on the back of the board and is labeled **LED** on the board silk. Cut the trace on this jumper to cut power to the "**on**" LED.

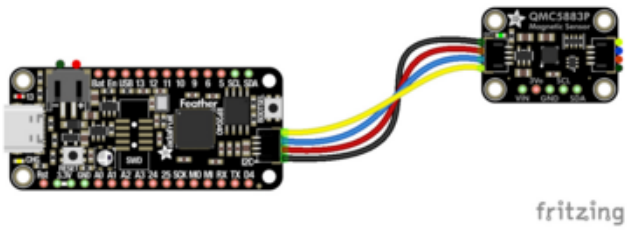
CircuitPython and Python

It's easy to use the **QMC5883P breakout** with Python or CircuitPython, and the Adafruit_CircuitPython_QMC5883P (https://adafru.it/1apf) module. This module allows you to easily write Python code to read magnetic field data from the sensor.

You can use this driver with any CircuitPython microcontroller board or with a computer that has GPIO and Python thanks to Adafruit_Blinka, our CircuitPython-for-Python compatibility library (https://adafru.it/BSN).

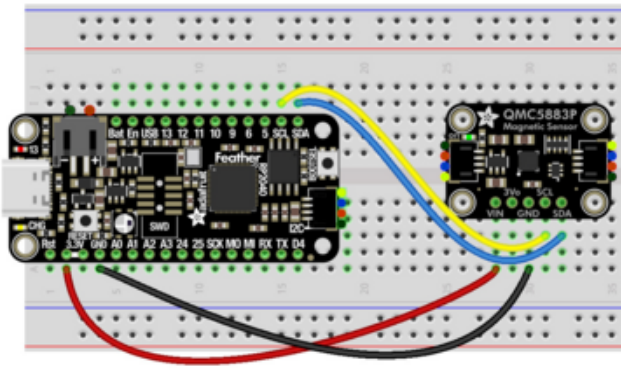
CircuitPython Microcontroller Wiring

First wire up the breakout to your board exactly as follows. The following is the breakout wired to a Feather RP2040 using the STEMMA connector:



Board STEMMA 3V to **breakout STEMMA VIN (red wire)**
Board STEMMA GND to **breakout STEMMA GND (black wire)**
Board STEMMA SCL to **breakout STEMMA SCL (yellow wire)**
Board STEMMA SDA to **breakout STEMMA SDA (blue wire)**

The following is the breakout wired to a Feather RP2040 using a solderless breadboard:

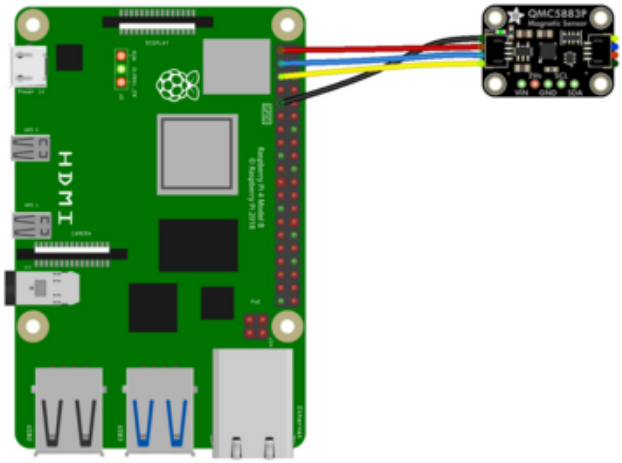


Board 3V to **breakout VIN (red wire)**
Board GND to **breakout GND (black wire)**
Board SCL to **breakout SCL (yellow wire)**
Board SDA to **breakout SDA (blue wire)**

Python Computer Wiring

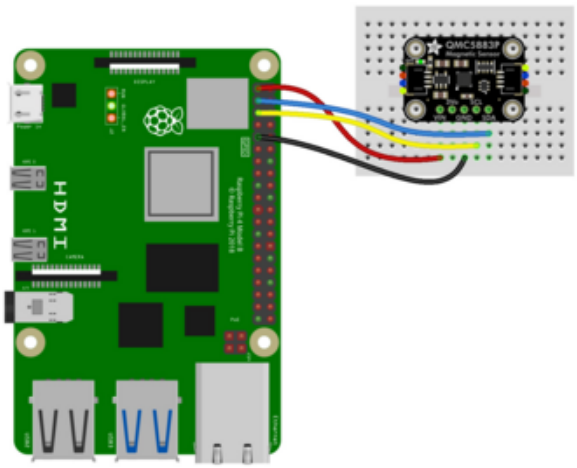
Since there are dozens of Linux computers/boards you can use, we will show wiring for Raspberry Pi. For other platforms, [please visit the guide for CircuitPython on Linux to see whether your platform is supported](https://adafru.it/BSN) (https://adafru.it/BSN).

Here's the Raspberry Pi wired with I2C using the STEMMA connector:



Pi 3V to **breakout STEMMA VIN (red wire)**
Pi GND to **breakout STEMMA GND (black wire)**
Pi SCL to **breakout STEMMA SCL (yellow wire)**
Pi SDA to **breakout STEMMA SDA (blue wire)**

Here's the Raspberry Pi wired with I2C using a solderless breadboard:



Pi 3V to **breakout VIN (red wire)**
Pi GND to **breakout GND (black wire)**
Pi SCL to **breakout SCL (yellow wire)**
Pi SDA to **breakout SDA (blue wire)**

Python Installation of QMC5883P Library

You'll need to install the **Adafruit_Blinka** library that provides the CircuitPython support in Python. This may also require enabling I2C on your platform and verifying you are running Python 3. Since each platform is a little different, and Linux changes often, please visit the CircuitPython on Linux guide to get your computer ready (https://adafru.it/BSN)!

Once that's done, from your command line run the following command:

- `pip3 install adafruit-circuitpython-qmc5883p`

If your default Python is version 3 you may need to run 'pip' instead. Just make sure you aren't trying to use CircuitPython on Python 2.x, it isn't supported!

CircuitPython Usage

To use with CircuitPython, you need to first install the **Adafruit_CircuitPython_QMC5883P** library, and its dependencies, into the **lib** folder on your **CIRCUITPY** drive. Then you need to update **code.py** with the example script.

Thankfully, we can do this in one go. In the example below, click the **Download Project Bundle** button below to download the necessary libraries and the **code.py** file in a zip file. Extract the contents of the zip file, and copy the **entire lib folder** and the **code.py** file to your **CIRCUITPY** drive.

Your **CIRCUITPY/lib** folder should contain the following folders and file:

- **adafruit_bus_device/**
- **adafruit_register/**
- **adafruit_qmc5883p.mpy**



Python Usage

Once you have the library `pip3` installed on your computer, copy or download the following example to your computer, and run the following, replacing **code.py** with whatever you named the file:

```
python3 code.py
```

Example Code

If running CircuitPython: Once everything is saved to the **CIRCUITPY** drive, [connect to the serial console](https://adafru.it/Bec) (https://adafru.it/Bec) to see the data printed out!

If running Python: The console output will appear wherever you are running Python.

```
# SPDX-FileCopyrightText: Copyright (c) 2025 Liz Clark for Adafruit Industries
#
# SPDX-License-Identifier: MIT

"""QMC5333P Simple Test"""

import time

import board

import adafruit_qmc5883p

i2c = board.I2C()
# i2c = board.STEMMA_I2C()

sensor = adafruit_qmc5883p.QMC5883P(i2c)

# configure sensor settings
# defaults to MODE_NORMAL, ODR_50HZ, RANGE_8G

# sensor.mode = adafruit_qmc5883p.MODE_CONTINUOUS
# sensor.data_rate = adafruit_qmc5883p.ODR_10HZ
# sensor.range = adafruit_qmc5883p.RANGE_2G

print("QMC5883P Magnetometer Test")
print("-" * 40)

while True:
    mag_x, mag_y, mag_z = sensor.magnetic

    print(f"X:{mag_x:2.3f}, Y:{mag_y:2.3f}, Z:{mag_z:2.3f} G")

    time.sleep(1)
```

First, the QMC5883P sensor is recognized over I2C. Then, in the loop, it prints out the scaled X, Y and Z readings in Gauss from the sensor.

```
Auto-reload is on. Simply save files over USB to run them or enter REPL to disable.
code.py output:
QMC5883P Magnetometer Test
----------------------------------------
X:-0.105, Y:-0.251, Z:0.467 G
X:-0.119, Y:-0.223, Z:0.450 G
X:-0.706, Y:-0.537, Z:-0.023 G
X:-0.855, Y:-0.416, Z:-0.153 G
X:-0.434, Y:-0.515, Z:-0.276 G
X:-0.613, Y:0.272, Z:-0.363 G
X:-0.699, Y:0.191, Z:-0.385 G
X:-0.574, Y:-0.546, Z:0.316 G
X:-0.659, Y:-0.666, Z:-0.173 G
X:-0.598, Y:-0.479, Z:0.442 G
X:-1.470, Y:-1.386, Z:-3.703 G
X:-0.509, Y:-0.674, Z:0.012 G
X:-0.753, Y:-0.718, Z:-0.444 G
X:-2.728, Y:-0.810, Z:-4.011 G
X:-0.510, Y:-0.667, Z:0.201 G
X:-0.612, Y:-0.205, Z:0.440 G
```

Python Docs

Python Docs (https://adafru.it/1ap6)

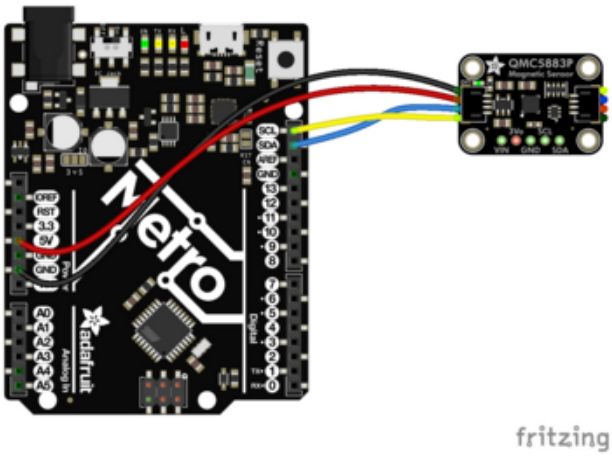
Arduino

Using the QMC5883P breakout with Arduino involves wiring up the breakout to your Arduino-compatible microcontroller, installing the Adafruit_QMC5883P (https://adafru.it/1ape) library, and running the provided example code.

Wiring

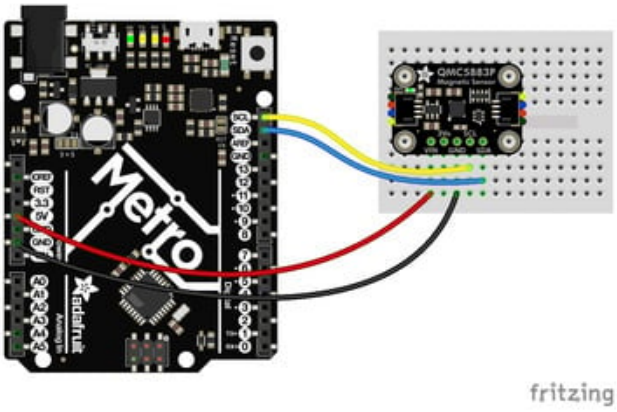
Wire as shown for a **5V** board like an Uno. If you are using a **3V** board, like an Adafruit Feather, wire the board's 3V pin to the sensor VIN.

Here is an Adafruit Metro wired up to the sensor using the STEMMA QT connector:



Board 5V to **breakout VIN (red wire)**
Board GND to **breakout GND (black wire)**
Board SCL to **breakout SCL (yellow wire)**
Board SDA to **breakout SDA (blue wire)**

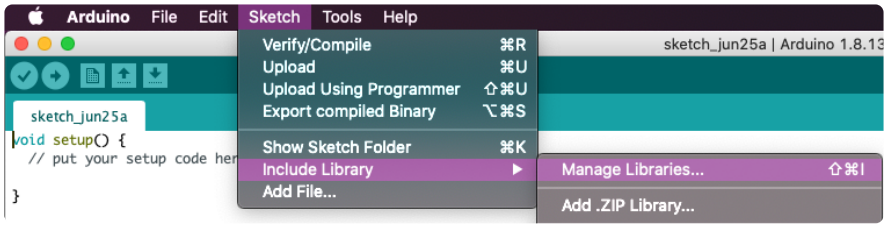
Here is an Adafruit Metro wired up using a solderless breadboard:



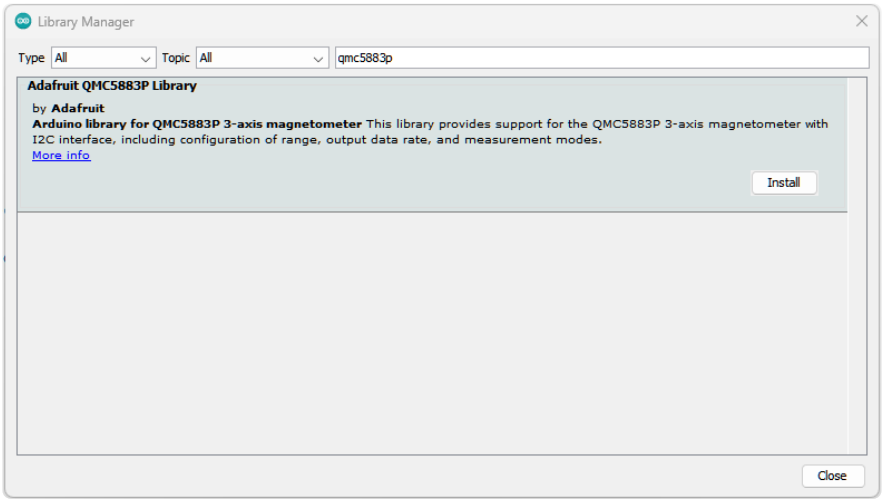
Board 5V to **breakout VIN (red wire)**
Board GND to **breakout GND (black wire)**
Board SCL to **breakout SCL (yellow wire)**
Board SDA to **breakout SDA (blue wire)**

Library Installation

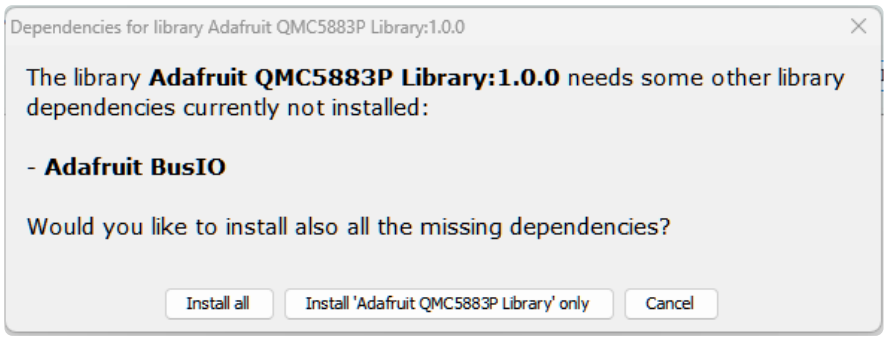
You can install the **Adafruit_QMC5883P** library for Arduino using the Library Manager in the Arduino IDE.



Click the **Manage Libraries ...** menu item, search for **Adafruit_QMC5883P**, and select the **Adafruit QMC5883P** library:



If asked about dependencies, click "Install all".



If the "Dependencies" window does not come up, then you already have the dependencies installed.

> If the dependencies are already installed, you must make sure you update them through the Arduino Library Manager before loading the example!

Example Code

```
/*
 * QMC5883P Test Sketch
 *
 * Basic test for the QMC5883P 3-axis magnetometer
 */

#include <Adafruit_QMC5883P.h>

Adafruit_QMC5883P qmc;

void setup() {
  Serial.begin(115200);
  while (!Serial)
    delay(10);

  Serial.println("QMC5883P Test");

  if (!qmc.begin()) {
    Serial.println("Failed to find QMC5883P chip");
    while (1)
      delay(10);
  }

  Serial.println("QMC5883P Found!");

  // Set to normal mode
  qmc.setMode(QMC5883P_MODE_NORMAL);

  qmc5883p_mode_t currentMode = qmc.getMode();
  Serial.print("Mode: ");
  switch (currentMode) {
    case QMC5883P_MODE_SUSPEND:
      Serial.println("Suspend");
      break;
    case QMC5883P_MODE_NORMAL:
      Serial.println("Normal");
      break;
    case QMC5883P_MODE_SINGLE:
```

```
    Serial.println("Single");
    break;
  case QMC5883P_MODE_CONTINUOUS:
    Serial.println("Continuous");
    break;
  default:
    Serial.println("Unknown");
    break;
}

// Set ODR (Output Data Rate) to 50Hz
qmc.setODR(QMC5883P_ODR_50HZ);
qmc5883p_odr_t currentODR = qmc.getODR();
Serial.print("ODR (Output Data Rate): ");
switch (currentODR) {
  case QMC5883P_ODR_10HZ:
    Serial.println("10Hz");
    break;
  case QMC5883P_ODR_50HZ:
    Serial.println("50Hz");
    break;
  case QMC5883P_ODR_100HZ:
    Serial.println("100Hz");
    break;
  case QMC5883P_ODR_200HZ:
    Serial.println("200Hz");
    break;
  default:
    Serial.println("Unknown");
    break;
}

// Set OSR (Over Sample Ratio) to 4
qmc.setOSR(QMC5883P_OSR_4);
qmc5883p_osr_t currentOSR = qmc.getOSR();
Serial.print("OSR (Over Sample Ratio): ");
switch (currentOSR) {
  case QMC5883P_OSR_8:
    Serial.println("8");
    break;
  case QMC5883P_OSR_4:
    Serial.println("4");
    break;
  case QMC5883P_OSR_2:
    Serial.println("2");
    break;
  case QMC5883P_OSR_1:
    Serial.println("1");
    break;
  default:
    Serial.println("Unknown");
    break;
}

// Set DSR (Downsample Ratio) to 2
qmc.setDSR(QMC5883P_DSR_2);
qmc5883p_dsr_t currentDSR = qmc.getDSR();
Serial.print("DSR (Downsample Ratio): ");
switch (currentDSR) {
  case QMC5883P_DSR_1:
    Serial.println("1");
    break;
  case QMC5883P_DSR_2:
    Serial.println("2");
    break;
  case QMC5883P_DSR_4:
    Serial.println("4");
    break;
  case QMC5883P_DSR_8:
```

```
      Serial.println("8");
      break;
    default:
      Serial.println("Unknown");
      break;
  }

  // Set Range to 8G
  qmc.setRange(QMC5883P_RANGE_8G);
  qmc5883p_range_t currentRange = qmc.getRange();
  Serial.print("Range: ");
  switch (currentRange) {
    case QMC5883P_RANGE_30G:
      Serial.println("±30G");
      break;
    case QMC5883P_RANGE_12G:
      Serial.println("±12G");
      break;
    case QMC5883P_RANGE_8G:
      Serial.println("±8G");
      break;
    case QMC5883P_RANGE_2G:
      Serial.println("±2G");
      break;
    default:
      Serial.println("Unknown");
      break;
  }

  // Set SetReset mode to On
  qmc.setSetResetMode(QMC5883P_SETRESET_ON);
  qmc5883p_setreset_t currentSetReset = qmc.getSetResetMode();
  Serial.print("Set/Reset Mode: ");
  switch (currentSetReset) {
    case QMC5883P_SETRESET_ON:
      Serial.println("Set and Reset On");
      break;
    case QMC5883P_SETRESET_SETONLY:
      Serial.println("Set Only On");
      break;
    case QMC5883P_SETRESET_OFF:
      Serial.println("Set and Reset Off");
      break;
    default:
      Serial.println("Unknown");
      break;
  }
}

void loop() {
  static int range_counter = 0;
  static unsigned long last_range_change = 0;

  // Change range every 3 seconds
  if (millis() - last_range_change > 3000) {
    qmc5883p_range_t ranges[] = {QMC5883P_RANGE_2G, QMC5883P_RANGE_8G,
                                 QMC5883P_RANGE_12G, QMC5883P_RANGE_30G};
    const char* range_names[] = {"±2G", "±8G", "±12G", "±30G"};

    qmc.setRange(ranges[range_counter]);
    Serial.print("*** Changed to range: ");
    Serial.println(range_names[range_counter]);

    range_counter = (range_counter + 1) % 4;
    last_range_change = millis();
    delay(100); // Let range change settle
  }

  if (qmc.isDataReady()) {
```

```
    int16_t x, y, z;
    float gx, gy, gz;

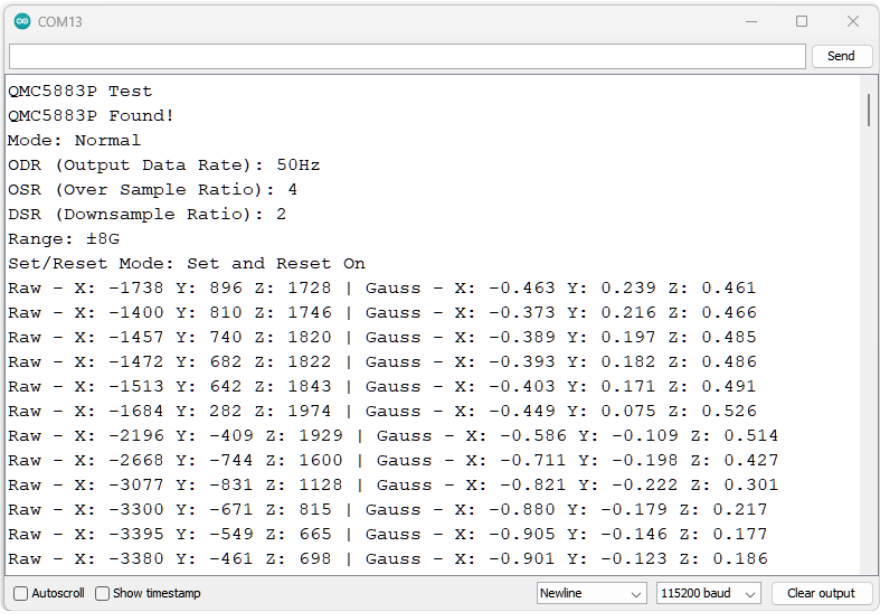
    if (qmc.getRawMagnetic(&x, &y, &z)) {
      Serial.print("Raw - X: ");
      Serial.print(x);
      Serial.print(" Y: ");
      Serial.print(y);
      Serial.print(" Z: ");
      Serial.print(z);

      if (qmc.getGaussField(&gx, &gy, &gz)) {
        Serial.print(" | Gauss - X: ");
        Serial.print(gx, 3);
        Serial.print(" Y: ");
        Serial.print(gy, 3);
        Serial.print(" Z: ");
        Serial.println(gz, 3);
      } else {
        Serial.println(" | Failed to convert to Gauss");
      }

      if (qmc.isOverflow()) {
        Serial.println("WARNING: Data overflow detected!");
      }
    } else {
      Serial.println("Failed to read magnetic data");
    }
  }

  delay(200);
}
```

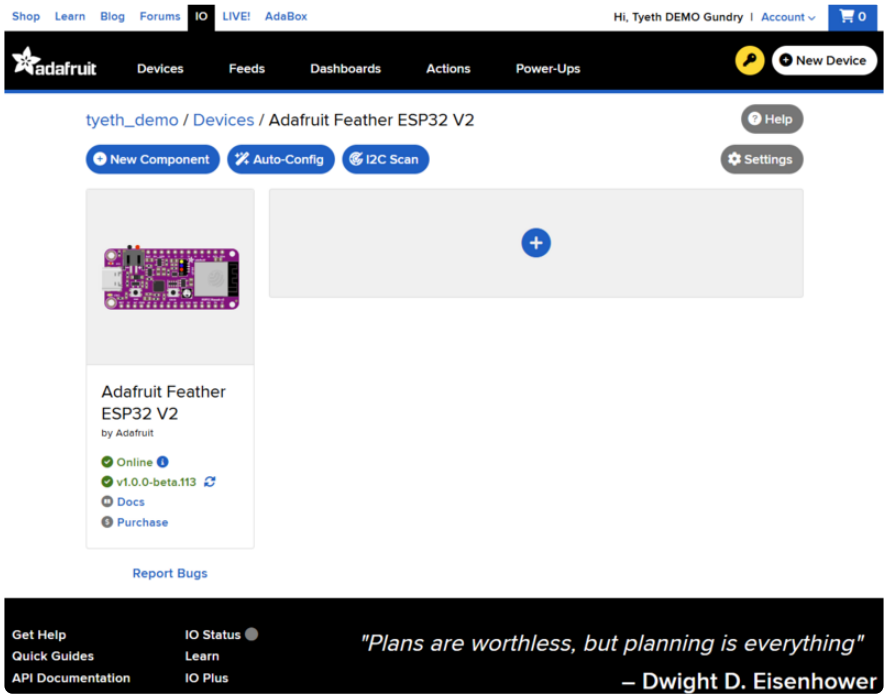
Upload the sketch to your board and open up the Serial Monitor (**Tools -> Serial Monitor**) at 115200 baud. You'll see the QMC5883P recognized over I2C. Then, the raw and scaled X, Y and Z readings will be printed out to the Serial Monitor.



Arduino Docs

Arduino Docs (https://adafru.it/1ap7)

WipperSnapper



What is WipperSnapper

WipperSnapper is a firmware designed to turn any WiFi-capable board into an Internet-of-Things device without programming a single line of code. WipperSnapper connects to Adafruit IO (https://adafru.it/fsU), a web platform designed (by Adafruit! (https://adafru.it/Bo5)) to display, respond, and interact with your project's data.

Simply load the WipperSnapper firmware onto your board, add credentials, and plug it into power. Your board will automatically register itself with your Adafruit IO account.

From there, you can add components to your board such as buttons, switches, potentiometers, sensors, and more! Components are dynamically added to hardware, so you can immediately start interacting, logging, and streaming the data your projects produce without writing code.

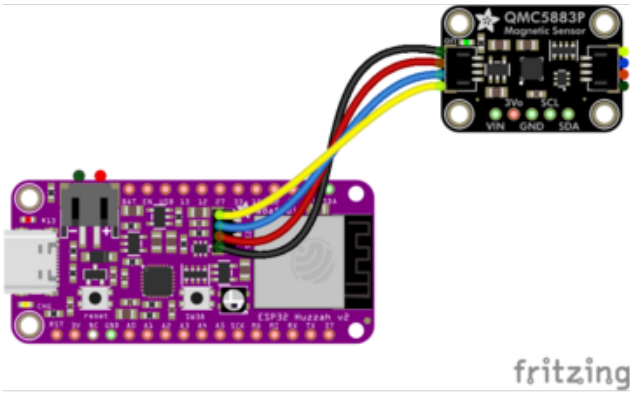
If you've never used WipperSnapper, click below to read through the quick start guide before continuing.

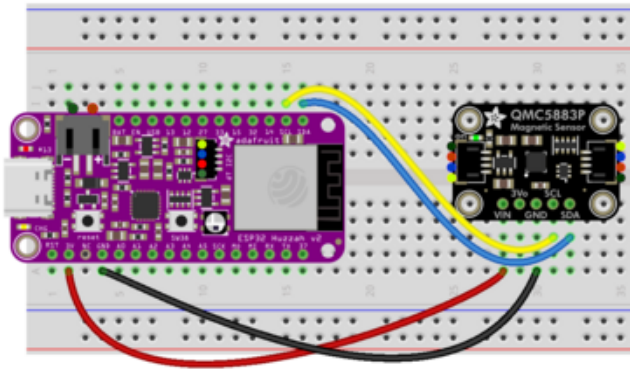
Quickstart: Adafruit IO WipperSnapper

https://adafru.it/Vfd

Wiring

First, wire up a QMC5883P to your board exactly as follows. Here is an example of the QMC5883P wired to an Adafruit ESP32 Feather V2 (http://adafru.it/5400) using I2C with a STEMMA QT cable (no soldering required) (http://adafru.it/4210)



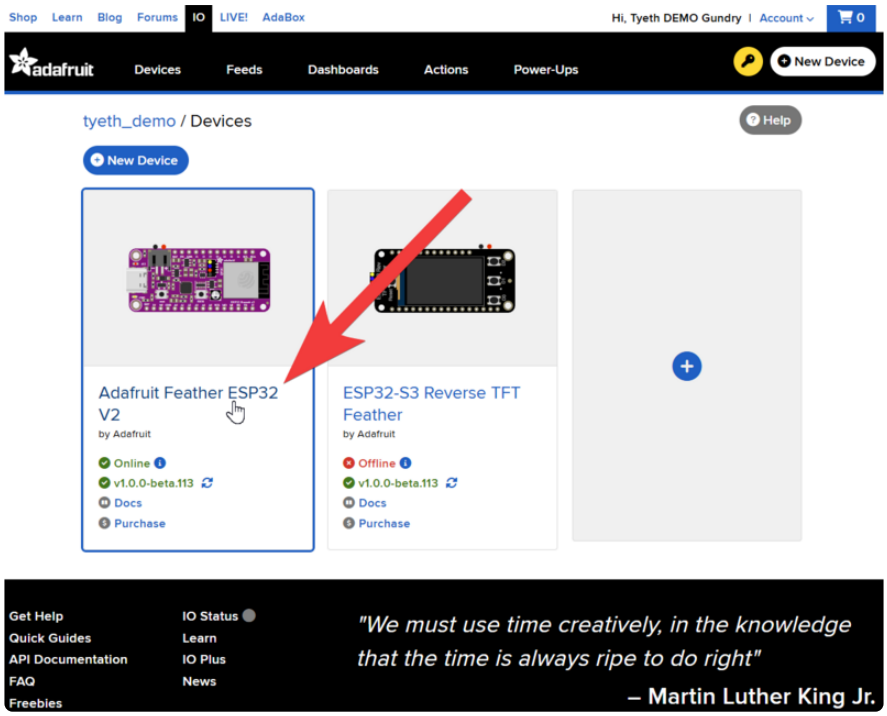


Board 3V to **sensor VIN (red wire on STEMMA QT)**
Board GND to **sensor GND (black wire on STEMMA QT)**
Board SCL to **sensor SCL (yellow wire on STEMMA QT)**
Board SDA to **sensor SDA (blue wire on STEMMA QT)**

Usage

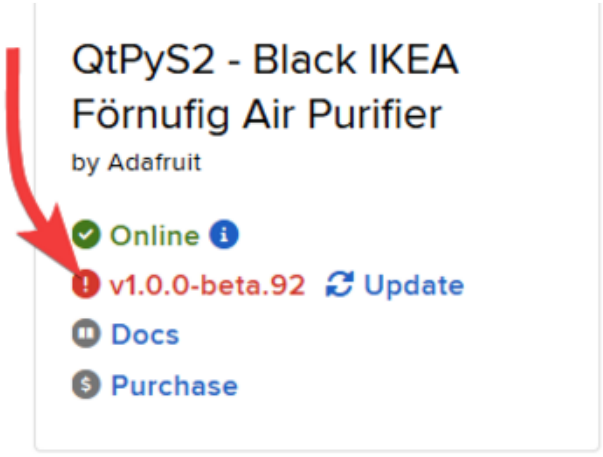
Connect your board to Adafruit IO Wippersnapper and navigate to the WipperSnapper board list **(https://adafru.it/TAu)**.

On this page, **select the WipperSnapper board you're using** to be brought to the board's interface page.



If you do not see your board listed here - you need [to connect your board to Adafruit IO (https://adafru.it/Vfd)](#) first.



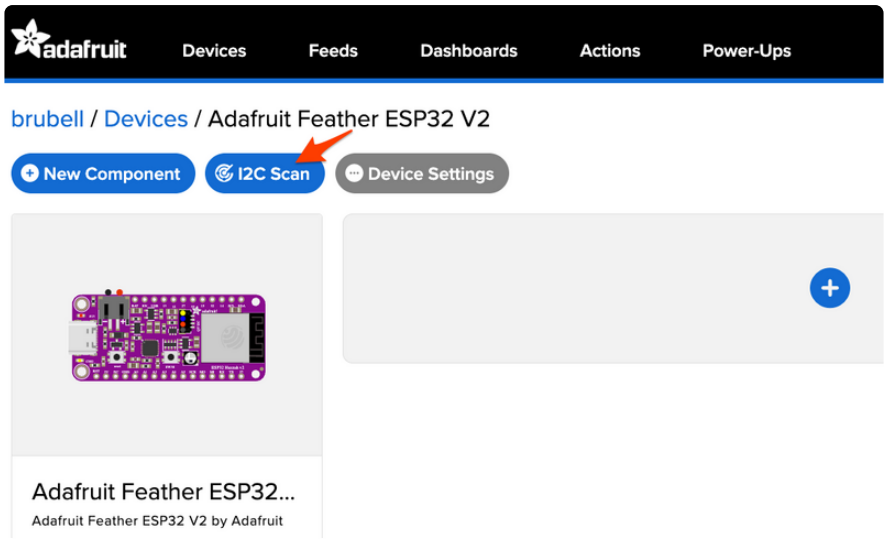


On the device page, quickly **check that you're running the latest version of the WipperSnapper firmware**.

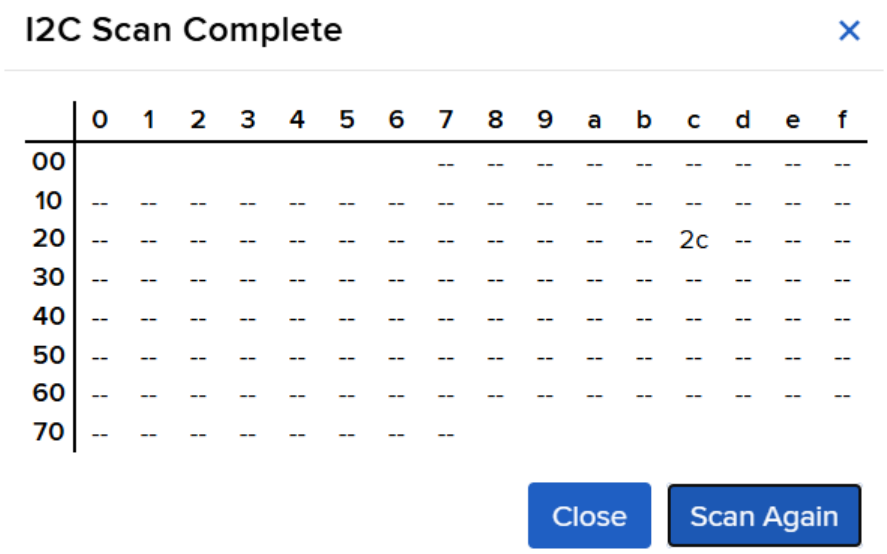
The device tile on the left indicates the version number of the firmware running on the connected board.

If the firmware version is green with a checkmark - continue with this guide.
If the firmware version is red with an exclamation mark "!" - [update to the latest WipperSnapper firmware (https://adafru.it/Vfd)](#) on your board before continuing.

Next, make sure the sensor is plugged into your board and click the **I2C Scan** button.



You should see the QMC5883P's default I2C address of `0x2C` pop-up in the I2C scan list.



I don't see the sensor's I2C address listed!

First, double-check the connection and/or wiring between the sensor and the board.

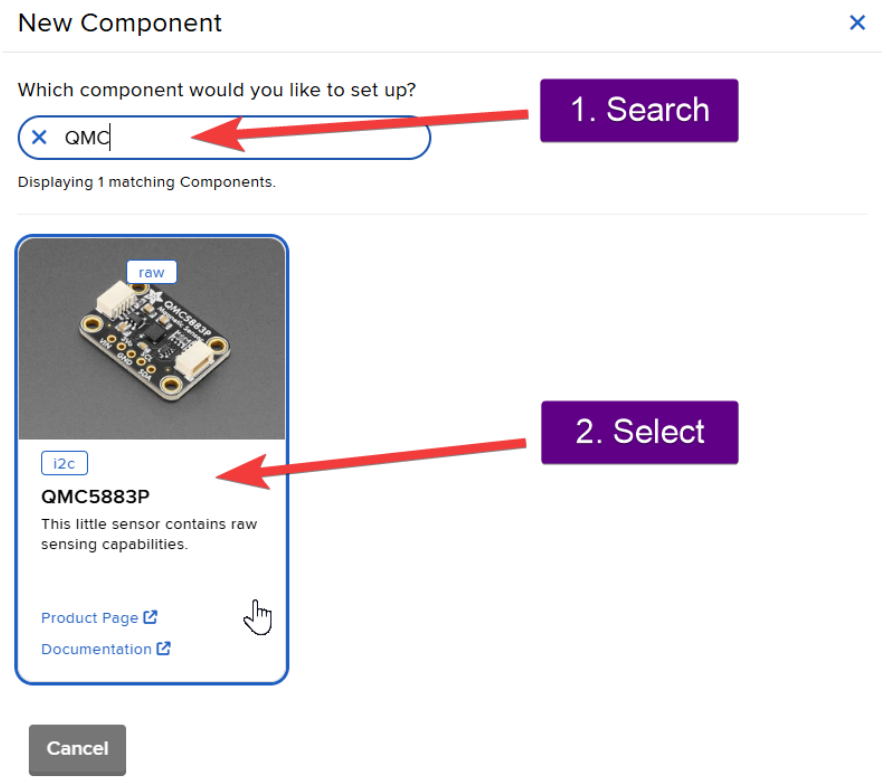
Then, reset the board and let it re-connect to Adafruit IO WipperSnapper.

With the sensor detected in an I2C scan, you're ready to add the sensor to your board.

Click the New Component button or the + button to bring up the component picker.



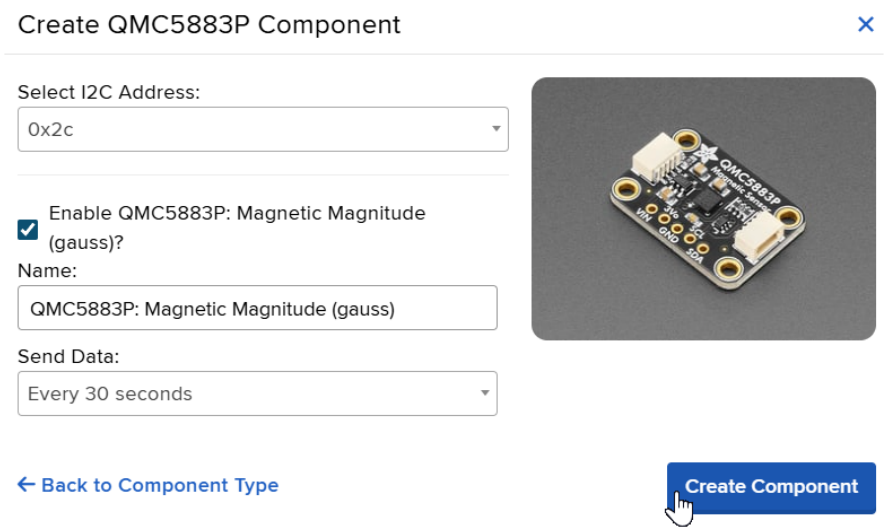
Adafruit IO supports a large amount of components. To quickly find your sensor, type `QMC` into the search bar, then select the **QMC5883P** component.



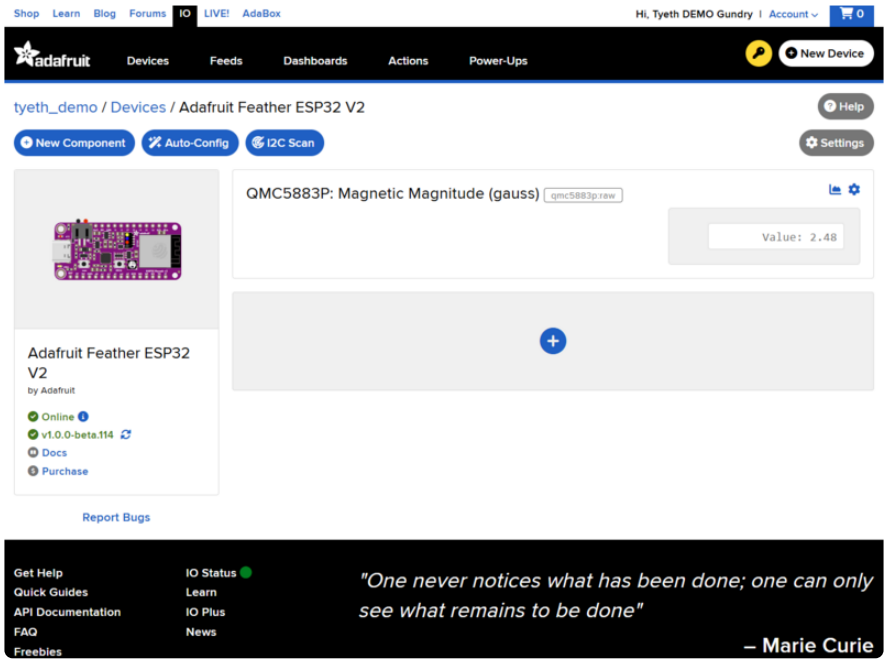
On the component configuration page, the QMC5883P's sensor address should be listed along with the sensor's settings.

The **Send Every** option is specific to each sensor's measurements. This option will tell the Feather how often it should read from the QMC5883P sensor and send the data to Adafruit IO. Measurements can range from every second to every 24 hours.

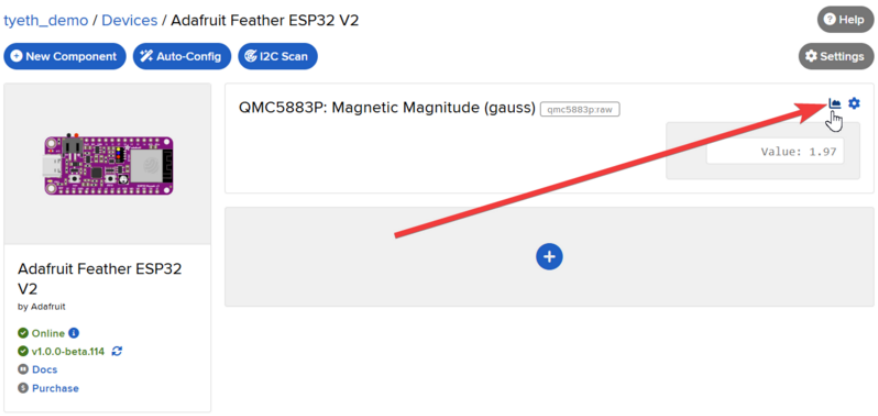
For this example, set the **Send Every** interval to every 30 seconds.



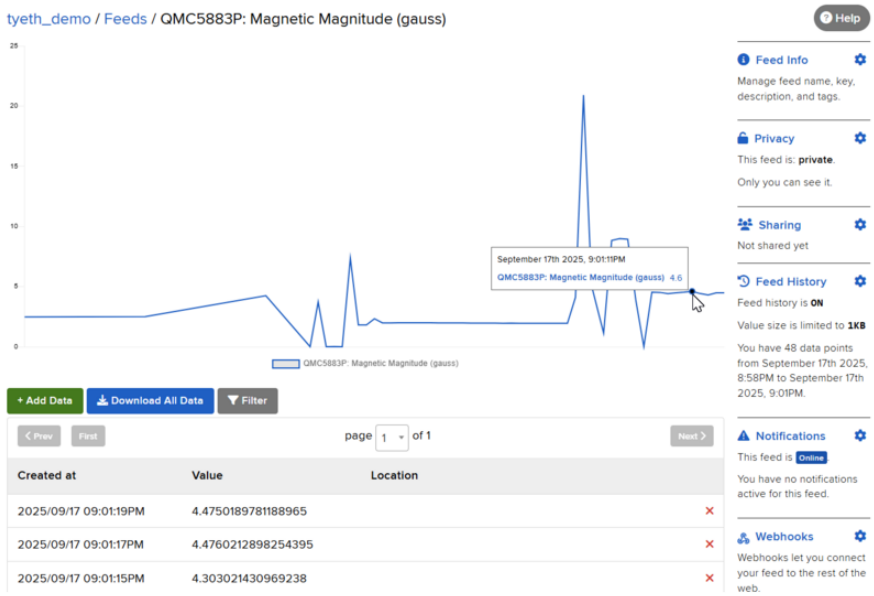
Your device interface should now show the sensor components you created. After the interval you configured elapses, WipperSnapper will automatically read values from the sensor(s) and send them to Adafruit IO.



To view the data that has been logged from the sensor, click on the graph next to the sensor name.



Here you can see the feed history and edit things about the feed such as the name, privacy, webhooks associated with the feed and more. If you want to learn more about how feeds work, [check out this page](https://adafru.it/10aZ).

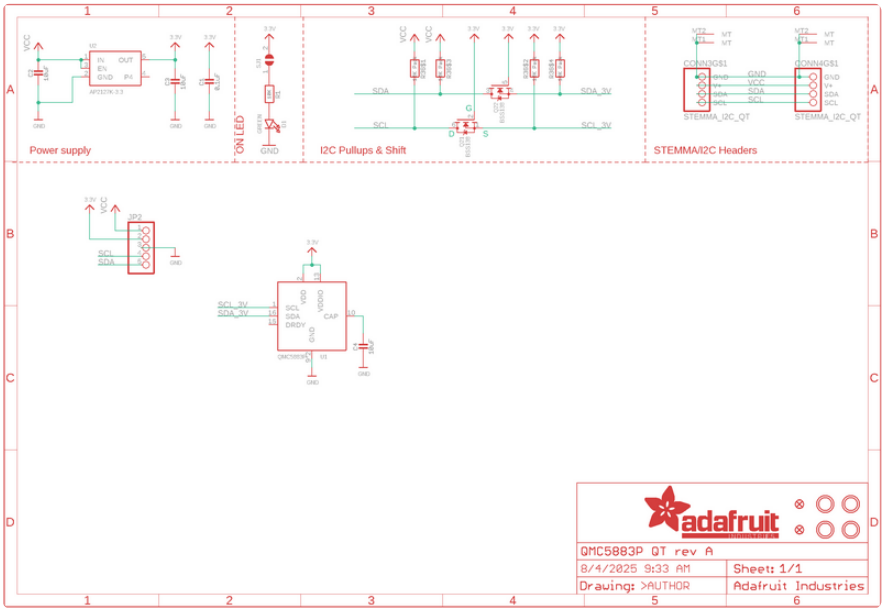


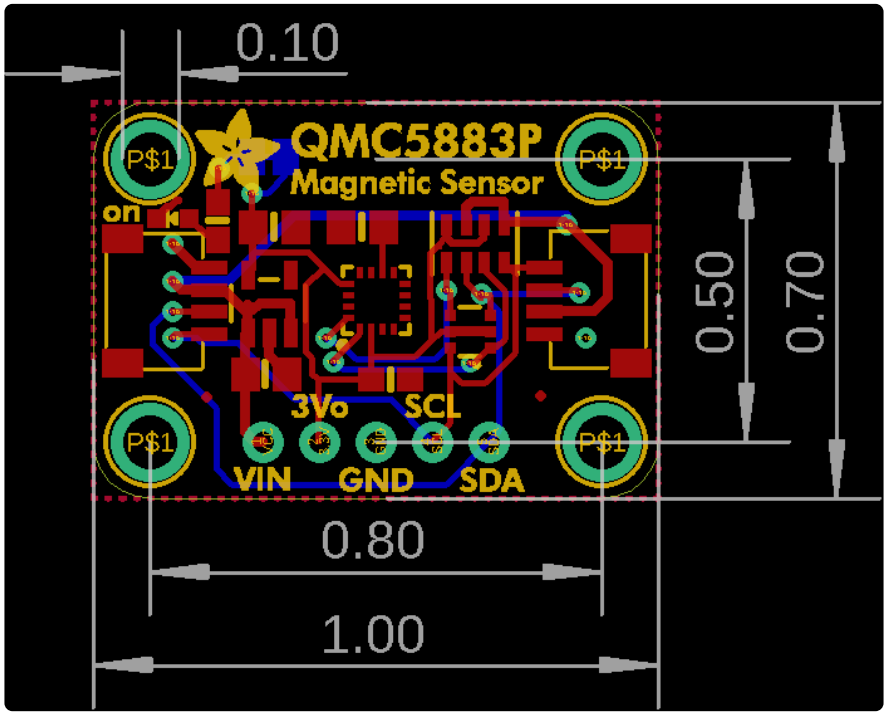
Downloads

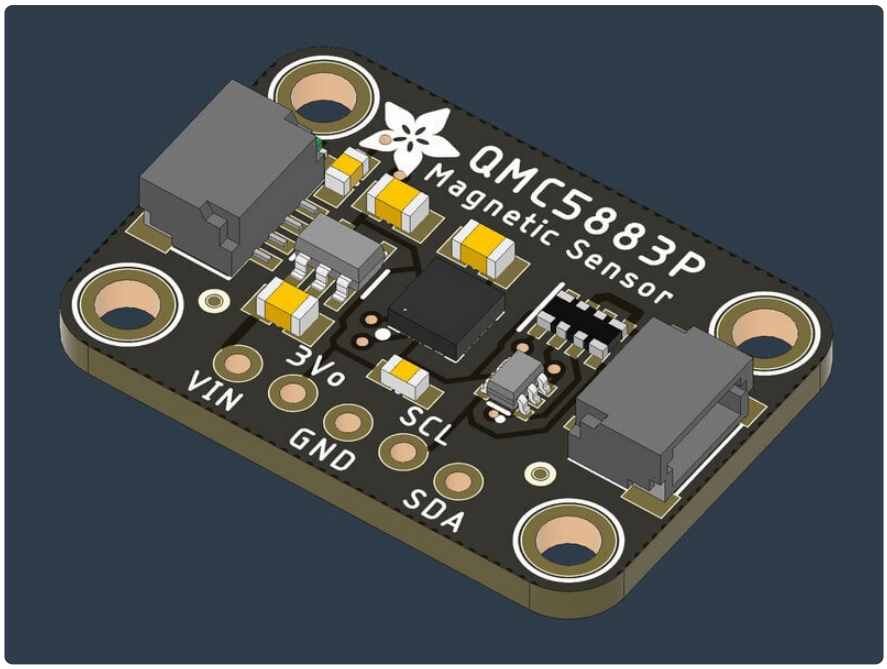
Files

- [QMC5883P Datasheet](http://adafru.it/63882847467)
- [EagleCAD PCB files on GitHub](https://adafru.it/1apg)
- [3D models on GitHub](https://adafru.it/1apS)
- [Fritzing object in the Adafruit Fritzing Library](https://adafru.it/1aph)

Schematic and Fab Print






QMC5883P
Magnetic Sensor
VIN
3Vo
GND
SCL
SDA